Unit III

Object-Oriented Analysis:

The analysis phase determines what the implementation must do, and the system design phase determines the plan of attack. The object design phase determines the full definitions of the classes and associations used in the implementation, as well as the interfaces and algorithms of the methods used to implement operations. The object design phase adds internal objects for implementation and optimizes data structures and algorithms.

Object-Oriented Language Features

Classes:

A class represents a real-world entity which acts as a blueprint for all the objects.

Objects:

Object-Oriented Programming system (OOPS) is designed based on the concept of **"**Object**".** It contains both variables (used for holding the data) and methods (used for defining the behaviours).

Data Abstraction:

Abstraction is a process where you show only "relevant" data and "hide" unnecessary details of an object from the user.

Encapsulation:

Encapsulation is the process of binding object state(fields) and behaviours (methods) together in a single entity called "Class".

Since it wraps both fields and methods in a class, it will be secured from the outside access.

We can restrict the access to the members of a class using access modifiers such as private, protected and public keywords.

Encapsulation helps us to achieve the re-usability of code without compromising the security.

Inheritance:

One class inherits or acquires the properties of another class.

Inheritance provides the idea of reusability of code and each subclass defines only those features that are unique to it, the rest of the features can be inherited from the parent class.

Inheritance is a process of defining a new class based on an existing class by extending its common data members and methods.

The parent class is called the base class or superclass. The child class that extends the base class is called the derived class or subclass or child class.

Polymorphism:

It is the concept where an object behaves differently in different situations.

Since the object takes multiple forms, it is called Polymorphism.

we can achieve it using method overloading and method overriding.

Passing Arguments To Methods

Every method has at least one argument, the implicit *self* argument. In a non-object-oriented language, the argument must be made explicit, of course. Methods may also have additional objects as arguments. Some of the arguments may be simple data values and not objects. In passing an object as an argument to a method, a reference to the object must be passed if the value of the object can be updated within the method. If the method is a query that simply extracts information from an object without modifying it, then a call-by-value mechanism can be used if the language permits. Passing all arguments as references is more uniform.

A consistent naming convention for method function names.

- in C, concatenate the class name, two underscores, and the operation name. (The two underscores separate the class name from the operation name, each of which may contain single underscores.)

Passing arguments in C:

In C, an object should always be passed by pointer. Although C permits structures to be passed by value, passing a pointer to an object structure is usually more efficient and provides a uniform access mechanism for both query and update operations.

```
Window__add_to_selections (self,
shape)

struct Window * self;

struct Shape * shape;
```

Structured Analysis and Structured Design (SA/SD)

Structured Analysis and Structured Design (SA/SD) is a diagrammatic notation which is design to help people understand the system. The basic goal of SA/SD is to improve quality and reduce the risk of system failure. It establishes a concrete management specification and documentation. It focuses on solidity, pliability and maintainability of the system.
Basically, the approach of SA/SD is based on the Data Flow Diagram. It is easy to understand SA/SD but it focuses on well-defined system boundary whereas the JSD approach is too complex and does not have any graphical representation.
SA/SD is combined known as SAD and it mainly focuses on following 3 points:

1. System
2. Process
3. Technology

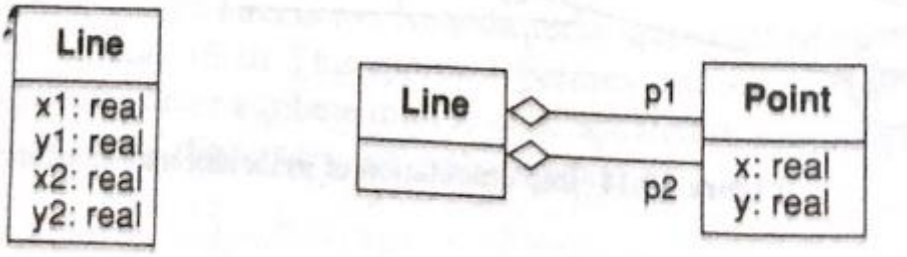
SA/SD involves 2 phases:

1. Analysis Phase: It uses Data Flow Diagram, Data Dictionary, State Transition diagram and ER diagram.
2. Design Phase: It uses the Structure Chart and Pseudo Code.

3. Object Representation

4. Implementing objects is mostly straightforward, but the designer must choose when to use primitive types in representing objects and when to combine groups of related objects. Classes can be defined in terms of other classes, but eventually, everything must be implemented in terms of built-in primitive data types, such as integers, strings, and enumerated types. For example, consider the implementation of a social security number within an employee object as shown in Figure:

5.
6. The social security number attribute can be implemented as an integer or a string or as an association to a social security number object, which itself can contain either an integer or a string. Defining a new class is more flexible but often introduces unnecessary indirection.
7. In a similar manner, the designer must often choose whether to combine groups of related objects.
8. Figure



9.
10. shows two common implementations of 2-dimensional lines, one as a separate class and one embedded as attributes within the Point class. Neither representation is inherently superior because both are mathematically correct.

Designing algorithms

Each operation specified in the functional model must be formulated as an algorithm. The analysis specification tells what the operation does from the viewpoint of its clients, but the algorithm shows how it is done. An algorithm may be subdivided into calls on simpler operations, and so on recursively, until the lowest-level operations are simple enough to implement directly without further refinement.

The algorithm designer must:

1. Choose algorithms that minimize the cost of implementing operations
2. Select data structures appropriate to the algorithms
3. Define new internal classes and operations as necessary
4. Assign responsibility for operations to appropriate classes

Design optimization

The basic design model uses the analysis model as the framework for implementation. The analysis model captures the logical information about the system, while the design model must add details to support efficient information access. The inefficient but semantically correct analysis model can be optimized to make the implementation more efficient, but an optimized system is more obscure and less likely to be reusable in another context. The designer must strike an appropriate balance between efficiency and clarity. During design optimization, the designer must:

- Add redundant associations to minimize access cost and maximize convenience
- Rearrange the computation for greater efficiency
- Save derived attributes to avoid recomputation of complicated expressions

Mapping object-oriented concepts using non-object-oriented language

Implementing an object-oriented design in a non-object-oriented language requires basically the same steps as implementing a design in an object-oriented language. The programmer using a non-object-oriented language must map object-oriented concepts into the target language, whereas the compiler for an object-oriented language performs such: napping automatically. The steps required to implement a design are:

- Translate classes into data structures
- Pass arguments to methods
- Allocate storage for objects
- Implement inheritance in data structures
- Implement method resolution
- Implement associations
- Deal with concurrency
- Encapsulate internal details of classes

Jackson Structured Development (JSD)

Jackson Structured Development (JSD) is another methodology, which has a different style than SA/SD or OMT. The JSD methodology was developed by Michael Jackson and is especially popular in Europe. JSD does not distinguish between analysis and design and instead lumps both phases together as specification. JSD divides system development into Two stages: specification, then implementation JSD first determines the "what" and then the "how." JSD is intended especially for applications in which timing is important.

JSD uses graphical models, as do SA/SD, OMT, and other techniques, but we will not show any JSD diagrams in this chapter. Sample diagrams are not required to communicate the flavour of JSD. In our opinion. JSD is less graphically oriented than SA/SD and OMT.

JSD is a **useful** methodology for the following types of applications:

- Concurrent software where processes must synchronize with each other.
- Real-time software. JSD modelling is extremely detailed and focuses on time.
- Microcode. JSD is thorough, makes no assumptions about the availability of an operating system, and considers concurrent processing and timing.
- Programming parallel computers. The JSD paradigm of many processes maybe helps here.

JSD is **ill-suited** for some other applications:

- High-level analysis. JSD does not foster a broad understanding of a problem. JSD is ineffective at abstraction and simplification. JSD meticulously handles details but does not help a developer grasp the essence of a problem.
- Databases. Database design is a more complex topic than Jackson implies. JSD modelling is biased towards actions and away from entities and attributes. As a natural consequence, it is a poor technique for database design.

- Conventional software running under an operating system. JSD's abstraction of hundreds or thousands of processes is confusing and unnecessary.

Combining three models

After analysis, we have the object, dynamic, and functional models, but the object model is the main framework around which the design is constructed. The object model from the analysis may not show operations. The designer must convert the actions and activities of the dynamic model and the processes of the functional model into operations attached to classes in the object model. In making this conversion, we begin the process of mapping the logical structure of the analysis model into a physical organization of a program.

Each state diagram describes the life history of an object. We can associate an operation with each event received by an object. Therefore the algorithm implementing an operation depends on the state of the object. If the same event can be received by more than one state of an object, then the code implementing the algorithm must contain a case statement dependent on the state.

An event sent by an object may represent an operation on another object. Events often occur in pairs, with the first event triggering an action and the second event returning the result or indicating the completion of that action. In this case, the event pair can be mapped into an operation performing the action and returning control provided that the events are on a single thread of control passing from object to object.

The flows in the diagram are intermediate values in the operation. The designer must convert the graph structure of the diagram into a linear sequence of steps in an algorithm. The processes in the data flow diagram constitute suboperations.

Physical packaging

Programs are made of discrete physical units that can be edited, compiled, imported, or otherwise manipulated. In some languages, such as C and Fortran, the units are source files. In Ada, the package is an explicit language construct for modularity. Object-oriented languages Aave various degrees of

packaging. In any large project, careful partitioning of implementation into packages (of whatever form) is important to permit different persons to cooperatively work on a program. Packaging involves the following issues

- Hiding internal information from outside view
- Coherence of entities
- Constructing physical modules

Documenting design considerations

The Design Document should be an extension of the Requirements Analysis Document. Again, this is a seamless process because the design uses the same notation as analysis but with more detail and specifics. It is particularly important to specify all operation interfaces by giving their arguments, results, input-output mappings, and side effects, If the Dynamic Model is implemented using an explicit state control or concurrent tasks, then the analysis model or its extension is adequate. Despite the seamless conversion from analysis to design, it is probably a good idea to keep the Design Document distinct from the Analysis Document.
Traceability from an element in the original analysis to the corresponding element in the design document should be straightforward since the design document is an evolution of the analysis model and retains the same names.

Implementation of control

The designer must refine the strategy for implementing the state-event models present in the dynamic model. As part of system design, you will have chosen a basic strategy for realizing the dynamic model. Now during object design, you must flesh out this strategy There are three basic approaches to implementing the dynamic model:

- Using the location within the program to hold state (procedure-driven system)
- Direct implementation of a state machine mechanism (event-driven system)
- Using concurrent tasks

Adjustment of inheritance

As object design progresses, the definitions of classes and operations can often be adjusted to increase the amount of inheritance. The designer should:

- Rearrange and adjust classes and operations to increase the inheritance
- Abstract common behaviour out of groups of classes
- Use delegation to share behaviour when inheritance is semantically invalid
- Procedural v/s OOP

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called functions.	In object-oriented programming, the program is divided into small parts called objects.
Procedural programming follows a top-down approach.	Object-oriented programming follows a bottom-up approach.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected etc.
Adding new data and function is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is less secure	Object-oriented programming provides data hiding so it is more secure.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, the function is more important than data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the unreal world.	Object-oriented programming is based on the real world.
Examples: C, FORTRAN, Pascal, Basic etc.	Examples: C++, Java, Python, C# etc.

Object-Oriented Programming Style

Reusability:

Reusable software reduces design, coding, and testing cost by amortizing effort over several designs. Reducing the amount of code also simplifies understanding, which increases the likelihood that the code is correct. Reuse is possible in conventional languages, but object-oriented languages greatly enhance the possibility of code reuse.

Extensibility:

Extensibility is a system design principle where the implementation takes future growth into consideration. It is a systemic measure of the ability to extend a system and the level of effort required to implement the extension. Extensions can be through the addition of new functionality or through modification of existing functionality. The central theme is to provide for change typically enhancements while minimizing impact to existing system functions.

Robustness

A method is robust if it does not fail even if it receives improper parameters. Robustness against internal bugs may be traded off against efficiency. Robustness against user errors should never be sacrificed.

Protect against errors. The software should protect itself against incorrect user input. Incorrect

user input should never cause a crash. Any method that accepts user input must validate input that could cause trouble.

Optimize after the program runs. Avoid optimizing more of the program than you have to, as optimization compromises extensibility, reusability, and understandability. If methods are properly encapsulated, they can be replaced with optimized versions without affecting the rest of the program.

Validate arguments. External operations, those available to users of the class, must rigorously check their arguments to prevent failure. But internal methods may assume their arguments are valid for efficiency reasons. Public methods must take more care to check the validity of arguments because external users are more likely to violate restrictions on arguments Internal, or private,

methods can often assume preconditions since the implementor has tighter control and can rely on the public methods that call them for error checking.
Avoid predefined limits. When possible use dynamic memory allocation to create data structures that do not have predefined limits.

Programming in the large

Programming-in-the-large refers to writing large, complex programs with teams of programmers. Human communication becomes paramount on such projects and requires proper software engineering practices. The following guidelines should be observed:

Do not prematurely begin programming.

Keep methods understandable.

Make methods readable Meaningful variable names increase readability.

Use exactly the same names as in the object model.

Choose the names carefully.

Use Programming guidelines.

Package into modules.

Document classes and methods.

Publish the specification.

Implementing Inheritance

There are several ways to implement data structures for inheritance in a non-object-oriented language:

- *Avoid it.* Many applications do not require inheritance. Many other applications have only a few classes requiring inheritance. Those classes not needing inheritance can be implemented as simple records.
- *Flatten the class hierarchy.* Use inheritance during design but expand each concrete class as an independent data structure during implementation. Each inherited operation must be reimplemented as a separate method on each concrete class. Flattening the hierarchy introduces duplication, but the use of language constructs such as C macros can help. One useful technique is to group some inherited

attributes into a record type and embed the record in each concrete class to reduce the number of duplicated lines in each declaration.

- *Break out separate objects*. Instead of inheriting common attributes from a superclass, a group of attributes can be pulled out of all the subclasses and implemented as a separate object with a reference to it stored within each subclass. Grouping attributes under a separate type permits a single method to be written to manipulate them. The subclasses must delegate operations to the referenced object.

Implementing Inheritance in C:

To handle single inheritance, embed the declaration for the superclass as the first part of each subclass declaration. The first field of each *struct* is a pointer to a class descriptor object shared by all direct instances of a class. The class descriptor object is a *struct* containing the class attributes, including the name of the class (optional) and the methods for the class.

The class Shape is an abstract class, with concrete subclasses *Box* and *Circle*. The C declarations for classes *Shape, Box*, and *Circle* are as follows:

```
struct Shape
{
struct ShapeClass * class;
Length x;
Length y:
};
struct Box
{
struct BoxClass * class;
Length x;
Length y;
Length width;
```

```
Length height;

};

struct Circle

{

struct CircleClass * class:

Length x;

Length y;

Length radius;

};
```

A pointer to a *Box* or *Circle* structure can be passed to a C function expecting a pointer to a *Shape* structure because the first part of the *Box* or *Circle* structure is identical to the *Shape* structure. (To be strictly correct, you should cast such an argument to be a pointer to *Shape*. but most compilers won't notice the difference anyway.) Because the prefix of the structure is the same as the superclass structure, the superclass method simply ignores the extra fields on the end. For example, a pointer to *Box* is interpreted as a pointer to *Shape* in the following call:

```
struct Box * box:

struct Window * window;

Window__add_to_selections (window,
box)
```

The first field of each structure is a pointer to the class descriptor for the actual class of each object instance. This field is needed only if the run-time method resolution is to be done.

Multiple inheritances cannot be implemented using this approach because a subclass which has two superclasses can align its attributes with both of them.

Implementing Associations

Implementing associations in a non-object-oriented language present the same two possibilities as in an object-oriented language: mapping them into pointers or implementing them directly as association container objects.

• **Mapping associations to pointers**. The traditional approach to implementing binary associations is to map each role of an association into an object pointer stored as a field of the
source object record. Each object contains a pointer to an associated object (if the multiplicity is "one" or "zero-one") or a pointer to a set of associated objects (if the multiplicity is greater than one). A set may be implemented using any appropriate available data structure often a linked-list or array, but a hash table or binary tree may be used when the greater execution efficiency warrants the extra programming effort.

The association may be implemented in one direction or in both directions. If it is traversed in only a single direction, it may be implemented as a pointer from one object to another.

• **Implementing association objects**. An association can be implemented directly as a data structure. If an association relates more than two classes then it cannot be mapped into pointers, and a separate object must be used in any case. In its basic form (and its mathematical definition), an association is simply a set of records, each containing one object ID for each associated class. The simplest approach is to implement an association as an array or list of records or record pointers. To traverse a binary link, the list is searched to find a link in which one field is equal to the source object, and the value of the other field is returned as the target object.

Implementing Association in C:

A binary association is usually implemented as a field in each associated object, containing a pointer to the related object or to an array of related objects. For example, the many-10-one association
between *Item* and *Group* would be implemented as:

```
struct Item

{

struct Itemclass * class;

struct Group * group;

};

struct Group

{

struct GroupClass * class;

int item_count;

struct Item ** Items;

};
```

Other data structures, such as a linked list or a hash table, can also be used to store sets of objects. In this example, a group is created from a set of selected items. The memory for the items pointer can be allocated all at once since the number of items in a group does not change. If it were possible to add a single new item to a group, then both pointers must be updated:

```
Group__add_item (self, item)

struct Group * self;

struct Item * item;

{

item->group = self;

self->items = (struct Item **) realloc (self->items, ++self->item count sizeof
Item *));

self->items [self->item_count-1] = item;

}
```

You can build more sophisticated data structures to avoid calling *realloc* more than necessary. It is useful to have available a library of generic container objects, such as variable-length arrays, lists, and hash tables.

Implementing Encapsulation

Encapsulation of data representation and method implementation is one of the major themes of object-oriented programming. Object-oriented languages provide constructs to encapsulate implementation. Some of this encapsulation is lost when the programmer must manually translate object-oriented concepts into a non-object-oriented language, but you can still take advantage of the encapsulation facilities provided by the language.

Encapsulation in C

C has a reputation for encouraging loose programming style harmful to encapsulation. Nevertheless, you can take the following steps to improve encapsulation:

- Avoid the use of global variables.
- Package the methods for each class into a separate file. Only include the declarations for classes whose internal structure you need, such as ancestor classes of the current class. Do not access the fields of objects of different classes; call an access method instead.
- Treat objects of other classes as type "void *"; while technically illegal, most compilers treat all pointers the same (but watch out for word length or alignment problems on some machines).
- Translating classes into data structures
- Normally you will implement each class as a single contiguous block of attributes-a record structure. Each attribute has a declared type, which can be a primitive type, such as integer, real or character or can be a structured value, such as an embedded record structure or a fixed-length array
- A variable that identifies an object must therefore be implemented as a sharable reference and not simply by copying the values of an object's attributes.
- **Translating Classes into C Struct Declarations**

- Each class in the design becomes a C *struct*. Each attribute defined in the class becomes a field of the C struct. The structure for the *Window* class is declared as:

```
struct Window
{
Length xmin;
Length ymin;
Length xmax;
Length ymax;
};
```

- *Length* is a C type (not a class) defined with a C typedef statement to provide greater modularity in the definition of values:

```
typedef float Length;
```

- In C, an object reference can be represented by a pointer to its object record:

```
struct Window * window;
Length x1 = window->xmin;
```

- An object can be allocated statically, automatically (on the stack), or dynamically (on the heap) because C can compose a pointer to any object, including one embedded within another structure.

Object-oriented design

The objects discovered during analysis serve as the skeleton of the design, but the object designer must choose among different ways to implement them with an eye toward minimizing execution time, memory, and other measures

of cost. In particular, the operations identified during the analysis must be expressed as algorithms, with complex operations decomposed into simpler internal operations. The classes, attributes, and associations from analysis must be implemented as specific data structures.

Steps of Object design:
During object design, the designer must perform the following steps:

1. *Combine the three models to obtain operations on classes*
2. *Design algorithms to implement operations*
3. *Optimize access paths to data*
4. *Implement control for external interactions*
5. *Adjust class structure to increase the inheritance*
6. *Design associations*
7. *Determine object representation*
8. *Package classes and associations into modules*